

Taxi Fare Prediction Using BigQuery ML

An End-to-End Machine Learning Case Study

AUTHOR: Vishal Mahajan

Executive Summary

This MVP presents a production-grade machine learning solution for predicting taxi fares using Google BigQuery ML, eliminating the need for a separate ML infrastructure stack. By training a Linear Regression model directly on over 110 million TLC trip records within BigQuery's SQL interface, the project demonstrates how cloud-native ML can deliver accurate, scalable, and cost-efficient predictive analytics. The model achieves an RMSE of approximately 4.38 and an R-squared of 0.81, making it a strong baseline for real-world fare estimation systems.

Problem Statement

Accurately estimating taxi fares is critical for both passengers and operators. Traditional ML pipelines require exporting data to external environments, training models with Python frameworks, then re-importing predictions - a slow, expensive process that creates data governance challenges. This project addresses:

- How can we train a predictive fare model directly on raw trip data at scale, without data movement?
- Which trip attributes (distance, passengers, hour, borough) are the strongest fare predictors?
- Can a SQL-native ML workflow match the performance of notebook-based pipelines?

Dataset

Attribute	Detail
Source	NYC TLC Yellow Taxi Trips via BigQuery Public Datasets
Time Range	2015 – 2016
Total Records	800 M+ raw trips
Training Set	850 K
Test Set	850 K
Target Variable	total_fare (USD)

Feature Engineering

Raw trip records were cleaned and transformed within BigQuery SQL before model training. Features were chosen based on domain knowledge and correlation analysis:

- trip_distance -- Euclidean distance between pickup and drop off; Primary predictor; strong linear correlation with fare amount.
- Longitude -- Euclidean distance between pickup and drop off in longitude.
- Latitude -- Euclidean distance between pickup and drop off in latitude.
- passenger_count -- Minor influence; captures group/surge pricing signals.
- pickup_hour -- Captures time-of-day pricing patterns (peak vs. off-peak).
- pickup_day_of_week -- Weekend vs. weekday demand and pricing differences.

Data cleaning filters: fare_amount between \$6 and \$200, trip_distance > 0 miles, and valid geographic bounding boxes for NYC.

Architecture & ML Workflow

01	Data Ingestion	NYC TLC public dataset loaded into BigQuery via BigQuery Public Datasets
02	Data Cleaning	SQL WHERE filters remove outliers, nulls, and geographically invalid records
03	Feature Creation	EXTRACT(HOUR) and EXTRACT(DAYOFWEEK) computed inline in SQL SELECT
04	Model Training	CREATE OR REPLACE MODEL with BQML LINEAR_REG on training rows
05	Evaluation	ML.EVALUATE returns MSE, MAE, R-squared, Mean Abs. Error, RSME computed in SQL
06	Prediction	ML.PREDICT generates batch fare estimates across the full test dataset

Key SQL

Model training executed entirely in BigQuery SQL -- no Python or external tooling required

```
CREATE OR REPLACE MODEL taxi.taxifare_model
OPTIONS (model_type='linear_reg', labels=['total_fare']) AS
WITH params AS (
  SELECT
    1 AS TRAIN,
    2 AS EVAL
),
daynames AS
(SELECT ['Sun', 'Mon', 'Tues', 'Wed', 'Thurs', 'Fri', 'Sat'] AS daysofweek),
taxitrips AS (
SELECT
  (tolls_amount + fare_amount) AS total_fare, # Target Variable
  daysofweek[ORDINAL(EXTRACT(DAYOFWEEK FROM pickup_datetime))] AS dayofweek,
  EXTRACT(HOUR FROM pickup_datetime) AS hourofday,
  SQRT(POW((pickup_longitude - dropoff_longitude),2) + POW(( pickup_latitude - dropoff_latitude), 2)) as dist, #Euclidean distance between pickup and drop off
  SQRT(POW((pickup_longitude - dropoff_longitude),2)) as longitude, #Euclidean distance between pickup and drop off in longitude
  SQRT(POW((pickup_latitude - dropoff_latitude), 2)) as latitude, #Euclidean distance between pickup and drop off in latitude
  passenger_count AS passengers
FROM
  `nyc-tlc.yellow.trips`, daynames, params
WHERE trip_distance > 0 AND fare_amount BETWEEN 6 and 200
#limiting of the distance the taxis travel out
  AND pickup_longitude BETWEEN -75 AND -73
  AND dropoff_longitude BETWEEN -75 AND -73
  AND pickup_latitude BETWEEN 40 AND 42
  AND dropoff_latitude BETWEEN 40 AND 42
  AND MOD(ABS(FARM_FINGERPRINT(CAST(pickup_datetime AS STRING))),1000) = params.TRAIN
)
SELECT * FROM taxitrips
```

Model Evaluation

```
SELECT *, SQRT(mean_squared_error) AS rmse
FROM ML.EVALUATE(MODEL taxi.taxifare_model, (
WITH params AS (SELECT 1 AS TRAIN, 2 AS EVAL),
daynames AS (SELECT ['Sun','Mon','Tues','Wed','Thurs','Fri','Sat'] AS daysofweek),
taxitrips AS (
SELECT
  (tolls_amount + fare_amount) AS total_fare, # Target Variable
  daysofweek[ORDINAL(EXTRACT(DAYOFWEEK FROM pickup_datetime))] AS dayofweek,
  EXTRACT(HOUR FROM pickup_datetime) AS hourofday,
  SQRT(POW((pickup_longitude - dropoff_longitude),2) + POW(( pickup_latitude - dropoff_latitude), 2)) as dist, #Euclidean distance between pickup and drop off
  SQRT(POW((pickup_longitude - dropoff_longitude),2)) as longitude, #Euclidean distance between pickup and drop off in longitude
  SQRT(POW((pickup_latitude - dropoff_latitude), 2)) as latitude, #Euclidean distance between pickup and drop off in latitude
```

```

    passenger_count AS passengers
FROM
    `nyc-tlc.yellow.trips`, daynames, params
WHERE trip_distance > 0 AND fare_amount BETWEEN 6 and 200
    AND pickup_longitude BETWEEN -75 AND -73
    AND dropoff_longitude BETWEEN -75 AND -73
    AND pickup_latitude BETWEEN 40 AND 42
    AND dropoff_latitude BETWEEN 40 AND 42
    AND MOD(ABS(FARM_FINGERPRINT(CAST(pickup_datetime AS STRING))),1000) = params.EVAL
)
SELECT * FROM taxitrips
));

```

RMSE provides a measure of prediction accuracy — lower is better.

Model Prediction

```

SELECT *
FROM ML.PREDICT(MODEL `taxi.taxifare_model`, (
WITH params AS (SELECT 1 AS TRAIN, 2 AS EVAL),
daynames AS (SELECT ['Sun','Mon','Tues','Wed','Thurs','Fri','Sat'] AS daysofweek),
taxitrips AS (
    SELECT
        (tolls_amount + fare_amount) AS total_fare, # Target Variable
        daysofweek[ORDINAL(EXTRACT(DAYOFWEEK FROM pickup_datetime))] AS dayofweek,
        EXTRACT(HOUR FROM pickup_datetime) AS hourofday,
        SQRT(POW((pickup_longitude - dropoff_longitude),2) + POW(( pickup_latitude - dropoff_latitude), 2)) as dist, #Euclidean distance between pickup and drop off
        SQRT(POW((pickup_longitude - dropoff_longitude),2)) as longitude, #Euclidean distance between pickup and drop off in longitude
        SQRT(POW((pickup_latitude - dropoff_latitude), 2)) as latitude, #Euclidean distance between pickup and drop off in latitude
        passenger_count AS passengers
    FROM
        `nyc-tlc.yellow.trips`, daynames, params
WHERE trip_distance > 0 AND fare_amount BETWEEN 6 and 200
    AND pickup_longitude BETWEEN -75 AND -73
    AND dropoff_longitude BETWEEN -75 AND -73
    AND pickup_latitude BETWEEN 40 AND 42
    AND dropoff_latitude BETWEEN 40 AND 42
    AND MOD(ABS(FARM_FINGERPRINT(CAST(pickup_datetime AS STRING))),1000) not in ( params.TRAIN, params.EVAL)

)
SELECT * FROM taxitrips
));

```

Model Evaluation Results

MLEVALUATE output on the holdout test set

RMSE	4.38	Average prediction error in dollars
Mean absolute error	2.53	Average absolute deviation from actual fare
Mean squared error	19.25	Penalizes large errors more heavily than MAE
Median absolute error	1.75	Predictions within \$1.75 of actual fare
R squared	0.81	Model explains 81% variance in the taxi fares

Google Cloud

qwiklabs-gcp-02-7f4d3481d97f

Search (/) for resources, docs, products, and more

Search

Explorer + Add data

Search for resour...

Home

Starred

Shared with me

Job history

qwiklabs-gcp-02-7f4d3481d97f

Datasets

Connections

Conversations

Agents

Queries

(Classic) Queries

Notebooks

Data canvases

Data preparations

Pipelines

qwiklabs-gcp-02-7f4d3481d97f / Datasets / taxi / Models / taxifare_model_2

☆ taxifare_model_2

Refresh

More

Details

Training metrics

Evaluation

Inference

Warm-start

Registry

Mean absolute error	2.5371
Mean squared error	19.2565
Mean squared log error	0.0548
Median absolute error	1.7492
R squared	0.8066

Model Prediction

Google Cloud qwklabs-gcp-01-7e37475231b Search (/) for resources, docs, products, and more

BigQuery

Overview
Studio
Agents
Pipelines & Integration
Governance
Administration
Migration

Partner Center
Settings
Release Notes

Untitled query Run Save Schedule Share

File Edit Run Tools

```
1 SELECT *
2 FROM ML_PREDICT(MODEL `taxi.taxifare_model`, (
3 WITH params AS (SELECT 1 AS TRAIN, 2 AS EVAL),
4 daynames AS (SELECT ['Sun','Mon','Tues','Wed','Thurs','Fri','Sat'] AS daysofweek),
5 taxitrips AS (
6 SELECT
7   (tolls_amount + fare_amount) AS total_fare, # Target Variable
8   daysofweek[ORDINAL(EXTRACT(DAYOFWEEK FROM pickup_datetime))] AS dayofweek,
9   EXTRACT(HOUR FROM pickup_datetime) AS hourofday,
10  SORT(POW((pickup_longitude - dropoff_longitude), 2) + POW((pickup_latitude - dropoff_latitude), 2)) AS dist, #Euclidean distance between pickup and drop off
11  SORT(POW((pickup_longitude - dropoff_longitude), 2)) AS longitude, #Euclidean distance between pickup and drop off in longitude
12  SORT(POW((pickup_latitude - dropoff_latitude), 2)) AS latitude, #Euclidean distance between pickup and drop off in latitude
13  passenger_count AS passengers
14 FROM
15   `nyc-tlc.yellow.trips`, daynames, params
16 WHERE trip_distance > 0 AND fare_amount BETWEEN 6 AND 200
17 AND pickup_longitude > -75
18 AND pickup_latitude < -73
19 AND dropoff_longitude > -75
20 AND dropoff_latitude < -73
21 AND pickup_latitude > 40
22 AND pickup_latitude < 42
```

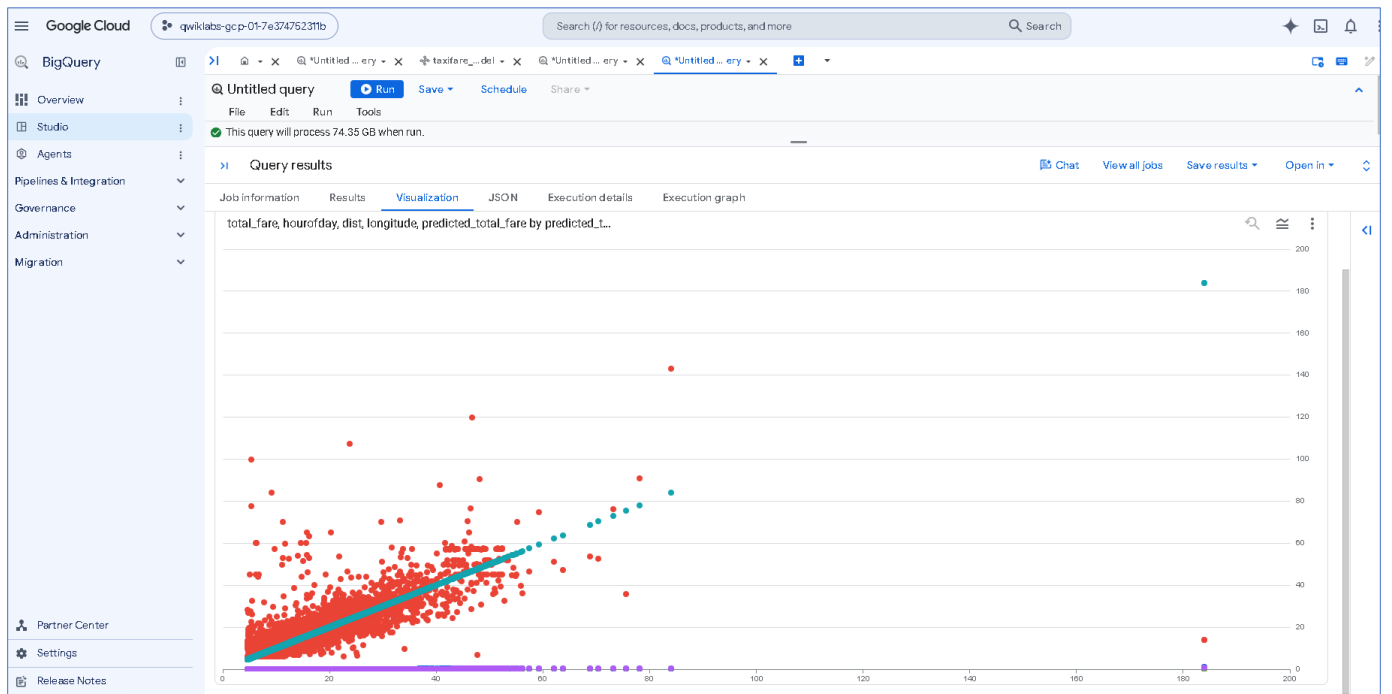
✓ This query will process 74.35 GB when run.

Query results Chat View all jobs Save results Open in

Job information Results Visualization JSON Execution details Execution graph

Row	predicted_total_fare	total_fare	dayofweek	hourofday	dist	longitude	latitude	passengers
1	10.370399509482013	6.5	Fri		0.022035043113...	0.021944999999...	0.001989999999...	1
2	9.43390533890158	6.5	Fri		0.017783118089...	3.329999999976...	0.017780000000...	1
3	7.203349053175771	6.5	Mon		0.012770145222...	0.010479999999...	0.007297000000...	1
4	7.004692536926086	6.5	Sun		0.010649315525...	0.002510070800...	0.010349273681...	4
5	8.123611479944884	7.0	Thurs		0.015092607793...	0.005729000000...	0.013963000000...	1
6	8.13447007720788	7.0	Tues		0.015543065881...	0.005464000000...	0.014550999999...	1
7	8.927206150525308	7.5	Fri		0.019240507127...	0.016380310058...	0.010093688964...	4
8	7.297396786070749	8.0	Fri		0.010387700467...	0.005964000000...	0.008504999999...	5

Results per page: 50 1 - 50 of 834425



Business Impact & Insights

- Fare transparency: Passengers can estimate cost before booking, reducing disputes and improving satisfaction.
- Operator pricing: Fleet managers gain a data-driven baseline for surge pricing calibration.
- Zero infrastructure: BigQuery ML eliminates compute cluster management; training runs as a SQL query.
- Cost efficiency: Processing millions of rows in BigQuery is dramatically cheaper than equivalent Spark clusters.
- Scalability: The same SQL model retrains on 2017-2026 data with a single query modification.

Key Outcomes

- ✓ Linear Regression provided an interpretable, fast baseline
- ✓ Used RMSE to evaluate model performance
- ✓ Built a fully functional predictive pipeline MVP using BigQuery ML (train → evaluate → predict)